



DSA Notes

Asymptotic Notations — Beginner's Cheatsheet



1. What are Asymptotic Notations?

Asymptotic Notations describe how an algorithm's performance grows as the input size (n) gets larger.

- Focus on **growth pattern** — not exact milliseconds
- They give us a **mathematical shorthand** to compare algorithms fairly
- Machine-independent — the notation stays the same on any computer



2. Three Types of Notation

O Big-O — Worst Case

The maximum time your algorithm will ever take. This is what we care about most — if your algorithm is fast in the worst case, it's fast everywhere.

Examples: $O(n)$, $O(n^2)$, $O(\log n)$

Ω Omega — Best Case

The minimum time — the absolute fastest your algorithm can finish. Useful to know, but rarely the focus in interviews or production.

Examples: $\Omega(1)$, $\Omega(n)$

Θ Theta — Exact / Average Case

A tight bound — used only when the best case and worst case are the same. Think of it as saying 'this algorithm always runs in exactly this time.'

Examples: $\Theta(n)$, $\Theta(n \log n)$

3. When to Use What?

Situation	Use Notation
Maximum / Worst-case time	Big-O (O)
Minimum / Best-case time	Omega (Ω)
Same behavior always (tight)	Theta (Θ)

 **In practice: Big-O is used 90% of the time. It's the industry default.**

4. Examples

Example 1: Loop that always runs

```
for (let i = 1; i <= n; i++) {  
  console.log(i);  
}  
  
// This loop ALWAYS runs n times.  
// No condition that could exit early.
```

Case	Notation	Value	
Best Case	$\Omega(n)$	n steps	✓ Defined
Worst Case	$O(n)$	n steps	✓ Defined
Theta	$\Theta(n)$	n steps	✓ Defined (best = worst)

 **Since best case = worst case (always n steps), Theta is defined here.**

Example 2: Condition-based (early exit possible)

```

if (n % 2 !== 0) return 0; // exits immediately if n is odd

for (let i = 0; i < n; i++) {
  console.log(i);
}

// If n is odd → returns in O(1).
// If n is even → runs full loop in O(n).

```

Case	Notation	Value	
Best Case	$\Omega(1)$	1 step	✓ Early exit
Worst Case	$O(n)$	n steps	✓ Full loop
Theta	$\Theta(?)$	—	✗ Not defined

👉 Theta cannot be defined here — behavior changes based on input. Best ≠ Worst.

📌 5. Key Rule for Theta

✓ Use Theta (Θ) ONLY when: Best Case = Worst Case

```

// Theta IS defined:
// Algorithm always takes n steps regardless of input

// Theta is NOT defined:
// Algorithm can exit early OR run longer based on conditions

```

- If there's an early **return** or **break** — Theta is likely undefined
- If the loop **always completes** the same number of steps — Theta is safe to use

📌 6. Growth Order Reminder

$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$
← FASTER			SLOWER →		

- Left side = faster algorithms = better performance
- Right side = slower — avoid $O(n^3)$ or worse for large inputs
- All three notations (O , Ω , Θ) apply across this entire spectrum

7. What to Focus On

✓ DO: Identify best and worst cases for every algorithm you write.

✓ DO: Use Big-O (worst case) as your default notation.

✓ DO: Use Theta when you can confirm best = worst case.

✗ IGNORE: Mathematical proofs and exact constants.

✗ IGNORE: Complex epsilon-delta style formal definitions.

8. Final Takeaway

Final Takeaway

- **Big-O** (O) → Worst case — maximum time. Use this most of the time.
- **Omega** (Ω) → Best case — minimum time. Rarely the focus.
- **Theta** (Θ) → Exact / tight bound — only when best = worst case.
- **In practice** → Focus mainly on Big-O. It's the industry standard.